Conferencia 4: listas

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-10-07

Hay que procesar las notas de un grupo. Con tres estudiantes se resuelve con lo que ya sabes:

```
nota1 = int(input("Nota: "))
nota2 = int(input("Nota: "))
nota3 = int(input("Nota: "))
```

Con treinta da vergüenza escribirlo, pero todavía se puede. El problema de verdad aparece cuando no sabes cuántos son. Un programa que sirva para cualquier grupo necesitaría tantas variables como estudiantes tenga el grupo, y las variables hay que escribirlas antes, cuando el programa se escribe, mucho antes de que nadie diga cuántos estudiantes hay. **No se puede teclear una cantidad de nombres que todavía no se sabe cuál es.**

Ese es el límite con el que cerró la clase pasada, y no se arregla con más variables ni con nombres más listos. Hace falta otra cosa: un solo nombre que guarde muchos valores, tantos como haga falta y decididos mientras el programa corre. Se llama **lista**, y es todo el contenido de hoy.

Mira además lo que se abre con eso. Hasta ahora un acumulador te daba el promedio de las notas, pero no la mediana, ni las notas ordenadas, ni cuántas están por encima del promedio, porque en cuanto leías la nota siguiente la anterior se perdía. Guardarlas todas es lo que permite mirarlas dos veces. Y al final de la clase, con eso solo, cae la criba de Eratóstenes.

1. La lista

Una lista es una secuencia de valores. Se escribe entre corchetes, con los elementos separados por comas:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas)
print(type(notas))
print(len(notas))
```

```
[4, 5, 3, 5, 2, 4]
<class 'list'>
6
```

`len` devuelve cuántos elementos tiene. Una lista puede estar vacía, y puede contener valores de cualquier tipo, incluso mezclados, aunque en la práctica casi siempre guardamos cosas del mismo tipo.

```
vacia = []
mezclada = [1, "dos", 3.0]
print(len(vacia), len(mezclada))
```

```
0 3
```

Los índices

A cada elemento se llega por su posición, que se llama **índice** y se escribe entre corchetes. **Los índices empiezan en cero.**

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[0])
print(notas[1])
print(notas[5])
```

```
4
5
4
```

Que empiecen en cero tiene una consecuencia que hay que tener presente todo el tiempo: el último índice válido de una lista de `n` elementos es `n - 1`, no `n`. Pedir uno más allá del final es un error:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[6])
```

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
IndexError: list index out of range
```

Ese `IndexError` lo vas a ver muchas veces este semestre, y casi siempre significa lo mismo: un ciclo que dio una vuelta de más.



Figura 1. Un solo nombre para seis casillas. Arriba el índice, que empieza en cero; abajo el índice negativo, que cuenta desde el final.

Python admite además índices negativos, que cuentan desde el final. `-1` es el último, `-2` el penúltimo. Es cómodo y evita escribir `notas[len(notas) - 1]`:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[-1])
print(notas[-2])
```

```
4
2
```

2. Recorrer una lista

Aquí es donde el `for` de la clase dos enseña por fin su verdadera cara. Hasta ahora lo usábamos siempre sobre `range`, para contar. Pero `for` no sirve para contar: sirve para **recorrer una secuencia**, y `range` era solo una secuencia de números.

```
notas = [4, 5, 3, 5, 2, 4]
```

```python
for nota in notas:
    print(nota, end=" ")
```

```
4 5 3 5 2 4
```

La variable `nota` toma en cada vuelta el valor de un elemento, en orden. No hay índices, no hay contador, no hay manera de pasarse del final. Esta es la forma normal de recorrer una lista en Python y debe ser tu primer instinto.

Con lo que ya sabes de acumuladores, calcular el promedio es inmediato:

```python
notas = [4, 5, 3, 5, 2, 4]
suma = 0

for nota in notas:
    suma += nota

print(suma / len(notas))
```

```
3.8333333333333335
```

Cuando hace falta el índice

A veces necesitas saber en qué posición vas, por ejemplo para imprimir un número de orden. Entonces se recorre `range(len(...))`:

```python
notas = [4, 5, 3, 5, 2, 4]

for i in range(len(notas)):
    print(f"Estudiante {i + 1}: {notas[i]}")
```

```
Estudiante 1: 4
Estudiante 2: 5
Estudiante 3: 3
Estudiante 4: 5
Estudiante 5: 2
Estudiante 6: 4
```

Y si necesitas las dos cosas, el índice y el elemento, Python tiene `enumerate`, que es más limpio que indexar a mano:

```python
notas = [4, 5, 3, 5, 2, 4]

for i, nota in enumerate(notas):
    print(f"Estudiante {i + 1}: {nota}")
```

```
Estudiante 1: 4
Estudiante 2: 5
Estudiante 3: 3
Estudiante 4: 5
```

```
Estudiante 5: 2
Estudiante 6: 4
```

Las dos variables separadas por coma tienen una explicación, que llega en la sección 6. Por ahora tómalo como la forma de escribirlo.

La regla: recorre con `for nota in notas` siempre que puedas, y usa el índice solo cuando de verdad lo necesites.

Buscar

El operador `in` dice si un valor está en la lista, y `count` cuántas veces:

```
notas = [4, 5, 3, 5, 2, 4]
print(5 in notas)
print(6 in notas)
print(notas.count(5))
print(notas.index(3))
```

```
True
False
2
2
```

`index` devuelve la posición de la primera aparición, y da error si el valor no está.

3. Modificar una lista

Todo lo que hemos visto hasta hoy era **inmutable**: no se puede cambiar un número ni una parte de un texto. Cuando escribías `x = x + 1` no modificabas el 5, creabas un 6 y le ponías el mismo nombre.

Las listas son distintas. Una lista **se puede modificar en el lugar**, y eso cambia cómo hay que pensar sobre ellas.

```
notas = [4, 5, 3, 5, 2, 4]
notas[2] = 5
print(notas)
```

```
[4, 5, 5, 5, 2, 4]
```

Las operaciones que más se usan:

OPERACIÓN	QUÉ HACE
`lista.append(x)`	agrega `x` al final
`lista.insert(i, x)`	inserta `x` en la posición `i`
`lista.pop()`	saca y devuelve el último
`lista.pop(i)`	saca y devuelve el de la posición `i`

OPERACIÓN	QUÉ HACE
`lista.remove(x)`	elimina la primera aparición de `x`
`lista.sort()`	ordena la lista en el lugar
`lista.reverse()`	la invierte en el lugar

```
notas = [4, 5, 3]
notas.append(2)
print(notas)

ultima = notas.pop()
print(ultima, notas)

notas.insert(0, 5)
print(notas)
```

```
[4, 5, 3, 2]
2 [4, 5, 3]
[5, 4, 5, 3]
```

`append` es la que más vas a usar, porque es como se construye una lista a partir de una vacía:

```
cuadrados = []

for i in range(1, 6):
    cuadrados.append(i * i)

print(cuadrados)
```

```
[1, 4, 9, 16, 25]
```

La sorpresa más cara del semestre

Presta atención a esto, porque es la fuente de más errores de este curso que cualquier otra cosa. Con los números, asignar copia el valor:

```
a = 5
b = a
a = 6
print(a, b)
```

```
6 5
```

`b` sigue valiendo 5, como trazamos en la primera clase. Ahora lo mismo con listas:

```
a = [1, 2, 3]
b = a
a.append(4)
print(a)
print(b)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

`b` cambió también, y nadie lo tocó. La razón es que `b = a` no copia la lista: copia el **nombre**. Después de esa línea hay una sola lista con dos nombres, y modificarla por cualquiera de los dos la modifica para ambos.

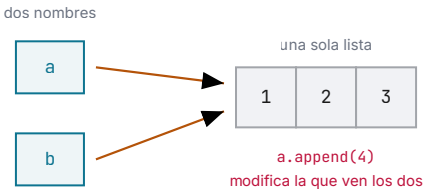


Figura 2. `b = a` no duplica las casillas. Duplica la flecha.

Esto no pasaba antes porque los números no se pueden modificar en el lugar; la única manera de cambiar `a` era asignarle otra cosa, y eso rompe el vínculo. Con una lista, `a.append(4)` no asigna nada: modifica el objeto que los dos nombres comparten.

Para copiar de verdad hay que pedirlo:

```
a = [1, 2, 3]
b = list(a)
a.append(4)
print(a)
print(b)
```

```
[1, 2, 3, 4]
[1, 2, 3]
```

Lo mismo pasa con los parámetros de una función. Si le pasas una lista a una función y la función la modifica, la lista de afuera queda modificada:

```
def agregar_cero(lista):
    lista.append(0)

notas = [4, 5, 3]
agregar_cero(notas)
print(notas)
```

```
[4, 5, 3, 0]
```

Compara con lo que viste la clase pasada, donde una función no podía cambiar el valor de un número de afuera. No es una contradicción: en los dos casos la función recibe el valor del argumento, pero el valor de una lista es la lista misma, no una copia de ella. A veces esto es exactamente lo que quieres. Cuando no lo es, la función debe trabajar sobre una copia y devolverla.

4. Rebanadas

Se puede pedir un pedazo de una lista con dos índices separados por dos puntos. El primero se incluye y el segundo no, la misma convención de `range`:

```
notas = [4, 5, 3, 5, 2, 4]
print(notas[1:4])
print(notas[:3])
print(notas[3:])
print(notas[::-1])
```

```
[5, 3, 5]
[4, 5, 3]
[5, 2, 4]
[4, 2, 5, 3, 5, 4]
```

Si omites el primer índice, empieza desde el principio; si omites el segundo, llega hasta el final. Un tercer número es el paso, y `-1` recorre al revés, que es la forma más corta de invertir una lista.

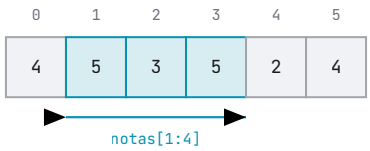


Figura 3. El primer índice entra y el segundo no, la misma convención de `range`.

Una rebanada siempre construye una **lista nueva**. Por eso `notas[:]` es otra manera de copiar, y por eso modificar la rebanada no toca el original:

```
notas = [4, 5, 3, 5, 2, 4]
copia = notas[:]
copia[0] = 100
print(notas)
print(copia)
```

```
[4, 5, 3, 5, 2, 4]
[100, 5, 3, 5, 2, 4]
```

5. El proyecto: las notas del grupo

Con esto ya podemos escribir el programa completo. Son tres piezas, cada una una función, como aprendimos la clase pasada.

Leer las notas hasta que el usuario escriba una línea en blanco:

```
def leer_notas():
    """Lee notas de la consola hasta una línea vacía. Devuelve la lista."""
    notas = []

    while True:
        linea = input("Nota (Enter para terminar): ")

        if linea == "":
```

```
            return notas

        notas.append(int(linea))
```

Calcular el promedio y el máximo. El máximo lo escribimos a mano, que es el patrón acumulador de la clase dos aplicado a una lista:

```
def promedio(notas):
    """Devuelve la media aritmética de una lista no vacía."""
    return sum(notas) / len(notas)

def maximo(notas):
    """Devuelve el mayor elemento de una lista no vacía."""
    mayor = notas[0]

    for nota in notas:
        if nota > mayor:
            mayor = nota

    return mayor

print(promedio([4, 5, 3, 5, 2, 4]))
print(maximo([4, 5, 3, 5, 2, 4]))
```

```
3.8333333333333335
5
```

Fíjate en que `maximo` empieza con `notas[0]` y no con cero. Empezar en cero funcionaría para notas, que son positivas, y fallaría con temperaturas bajo cero. Empezar por el primer elemento es siempre correcto, y a cambio la función exige que la lista no esté vacía.

Ahora que lo escribiste, te digo que Python ya lo trae: `max`, `min` y `sum` hacen esto mismo. Úsalos de aquí en adelante, pero el ejercicio valía la pena, porque en el examen te van a pedir el segundo mayor, o el mayor que cumple una condición, y ahí no hay función que lo haga por ti.

```
notas = [4, 5, 3, 5, 2, 4]
print(sum(notas), max(notas), min(notas), len(notas))
```

```
23 5 2 6
```

Lo que antes no se podía

Con las notas guardadas, aparecen las preguntas que la clase dos no podía responder. Cuántas están por encima del promedio, que exige recorrer la lista dos veces:

```
notas = [4, 5, 3, 5, 2, 4]
media = sum(notas) / len(notas)
sobre_la_media = 0

for nota in notas:
    if nota > media:
        sobre_la_media += 1

print(f"Promedio {media:.2f}, y {sobre_la_media} notas por encima")
```

```
Promedio 3.83, y 4 notas por encima
```

Y la mediana, que es el valor del medio una vez ordenadas:

```python
def mediana(notas):
    """Devuelve la mediana de una lista no vacía."""
    ordenadas = sorted(notas)
    mitad = len(ordenadas) // 2

    if len(ordenadas) % 2 == 1:
        return ordenadas[mitad]

    return (ordenadas[mitad - 1] + ordenadas[mitad]) / 2

print(mediana([4, 5, 3, 5, 2, 4]))
print(mediana([4, 5, 3]))
```

```
4.0
4
```

`sorted` devuelve una lista nueva ordenada, y no toca la original; `lista.sort()` ordena en el lugar y no devuelve nada. Hoy los usamos como cajas negras. Dentro de tres semanas vamos a escribir nosotros el algoritmo que hay dentro.

6. Tuplas

Python tiene otra secuencia que se parece mucho a la lista. Se llama **tupla** y se escribe con paréntesis en lugar de corchetes:

```python
punto = (3, 4)
print(punto)
print(type(punto))
print(punto[0], punto[1], len(punto))
```

```
(3, 4)
<class 'tuple'>
3 4 2
```

Se indexa, se rebana y se recorre con `for` igual que una lista. La diferencia es una sola, y es la de la sección 3: **una tupla no se puede modificar.**

```python
punto = (3, 4)
punto[0] = 10
```

```
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
TypeError: 'tuple' object does not support item assignment
```

No tiene `append`, ni `pop`, ni `sort`. Una vez construida se queda como está, y por eso la sorpresa de `b = a` no puede pasar con una tupla: los dos nombres comparten el mismo objeto, pero ninguno de los dos lo puede cambiar.

Para qué sirve una lista que no se puede cambiar

Una lista guarda muchos valores, cuántos no se sabe de antemano, y todos significan lo mismo: todos son notas. Una tupla guarda unos pocos valores, siempre la misma cantidad, y cada posición significa algo distinto. En `(3, 4)` el primero es la x y el segundo la y. En `("Ana", 5)` el primero es un nombre y el segundo una nota. Agregarle un tercer elemento a un punto del plano no tiene sentido, y la tupla no deja hacerlo.

Las dos cosas se combinan. Un grupo es una lista, porque no se sabe cuántos estudiantes tiene, y cada estudiante es una tupla, porque siempre es un nombre y una nota:

```
grupo = [("Ana", 5), ("Luis", 3), ("Marta", 4)]

for estudiante in grupo:
    print(estudiante[0], "sacó", estudiante[1])
```

```
Ana sacó 5
Luis sacó 3
Marta sacó 4
```

Desempaquetar

Los índices `[0]` y `[1]` funcionan, pero obligan a recordar qué había en cada posición. Python permite repartir una tupla entre varias variables de un golpe, una por posición:

```
punto = (3, 4)
x, y = punto
print(x)
print(y)
```

```
3
4
```

Eso se llama **desempaquetar**. A la izquierda tiene que haber tantas variables como elementos tenga la tupla:

```
x, y = (3, 4, 5)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 2, got 3)
```

Y se puede desempaquetar en el mismo `for`, que es como se recorre de verdad una lista de tuplas:

```
grupo = [("Ana", 5), ("Luis", 3), ("Marta", 4)]

for nombre, nota in grupo:
    print(nombre, "sacó", nota)
```

```
Ana sacó 5
Luis sacó 3
Marta sacó 4
```

Con esto ya se puede leer lo que hacía `enumerate` en la sección 2. `enumerate` produce tuplas de dos elementos, la posición y el valor, y `for i, nota in enumerate(notas)` desempaqueta cada una:

```
notas = [4, 5, 3]
print(list(enumerate(notas)))
```

```
[(0, 4), (1, 5), (2, 3)]
```

La coma es la que hace la tupla

Los paréntesis son casi siempre opcionales. Lo que construye la tupla es la coma:

```
t = 3, 4
print(t, type(t))

u = (5)
v = (5,)
print(type(u), type(v))
```

```
(3, 4) <class 'tuple'>
<class 'int'> <class 'tuple'>
```

`(5)` es el número 5 entre paréntesis, como en `(5 + 1) * 2`. Una tupla de un solo elemento necesita la coma al final.

De ahí salen dos cosas que vas a usar todo el tiempo. La primera es que una función puede devolver varios valores, porque `return a, b` devuelve una tupla, y quien la llama la desempaqueta:

```
def minimo_y_maximo(notas):
    """Devuelve el menor y el mayor elemento de una lista no vacía."""
    return min(notas), max(notas)

menor, mayor = minimo_y_maximo([4, 5, 3, 5, 2, 4])
print(menor, mayor)
```

```
2 5
```

La segunda es el intercambio de dos variables. Escribir `a = b` y después `b = a` no sirve, porque la primera línea pierde el valor viejo de `a`. Con tuplas sale en una línea:

```
a = 1
b = 2
a, b = b, a
print(a, b)
```

```
2 1
```

La derecha se evalúa primero y construye la tupla `(2, 1)` con los valores viejos; después la izquierda la desempaqueta. Te va a servir en el ejercicio 3.

7. Listas de listas

Un elemento de una lista puede ser cualquier cosa, incluso otra lista. Así se representa una tabla: las notas de varios estudiantes en varias asignaturas, una lista por estudiante.

```
tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]

print(len(tabla))
print(tabla[0])
print(tabla[0][2])
```

```
3
[5, 4, 5]
5
```

`tabla[0]` es la primera fila, que es una lista. `tabla[0][2]` es la tercera nota de esa fila. El primer índice escoge la fila y el segundo la columna.

Para recorrerla hacen falta dos ciclos, uno dentro de otro:

```
tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]

for fila in tabla:
    for nota in fila:
        print(nota, end=" ")
    print()
```

```
5 4 5
3 3 4
2 5 3
```

El `print()` sin argumentos al final del ciclo exterior es lo que salta de línea al terminar cada fila. Quítalo y verás las nueve notas en una sola línea.

Combinando lo de hoy, el promedio de cada estudiante sale en cuatro líneas:

```
tabla = [
    [5, 4, 5],
    [3, 3, 4],
    [2, 5, 3],
]
```

```python
for i, fila in enumerate(tabla):
    print(f"Estudiante {i + 1}: promedio {sum(fila) / len(fila):.2f}")
```

```
Estudiante 1: promedio 4.67
Estudiante 2: promedio 3.33
Estudiante 3: promedio 3.33
```

Construir una matriz, y una trampa

Para hacer una tabla de ceros uno intenta lo obvio, y lo obvio está mal:

```python
tabla = [[0] * 3] * 2
print(tabla)

tabla[0][0] = 9
print(tabla)
```

```
[[0, 0, 0], [0, 0, 0]]
[[9, 0, 0], [9, 0, 0]]
```

Cambiamos una casilla y se cambiaron dos. Es exactamente el problema de la sección 3: `[fila] * 2` no hace dos filas, hace **una sola fila con dos nombres**. La forma correcta es construir cada fila por separado:

```python
tabla = []

for i in range(2):
    tabla.append([0] * 3)

tabla[0][0] = 9
print(tabla)
```

```
[[9, 0, 0], [0, 0, 0]]
```

`[0] * 3` sí está bien, porque los ceros son inmutables y no importa que se compartan. Lo que no se puede repetir con `*` es una lista que después vas a modificar.

8. La criba de Eratóstenes

La clase pasada quedó planteado un problema y no se pudo resolver. Recuérdalo: para imprimir los primos hasta `n`, `es_primo` los interroga de uno en uno, y para decidir si 1009 es primo no usa nada de lo que averiguó sobre los mil anteriores. Eratóstenes hace lo contrario. En lugar de preguntar número por número, **tacha.** El 2 es primo, y de un tirón se tachan todos sus múltiplos; el siguiente sin tachar es el 3, que por eso mismo es primo, y se tachan los suyos; después el 5:



Figura 4. Cada color es el primo que tachó esa casilla, y no hubo una sola división. El reparto es muy desparejo: de las 15 casillas tachadas, el 2 tacha 11, el 3 tacha 3 y el 5 tacha 1. El primer primo hace casi todo el trabajo, y de ahí sale lo barata que es la criba.

Lo que faltaba era dónde anotar los tachones: un sí o un no por cada número entre 2 y `n`, con `n` decidido por quien usa el programa. Eso ya es una sola línea:

```
n = 25
compuesto = [False] * (n + 1)

print(len(compuesto))
print(compuesto[0], compuesto[25])
```

```
26
False False
```

Veintiséis casillas, todas en `False`, y el 26 salió de una variable. Con casillas de la 0 a la `n`, el número `k` vive en la casilla `k`; cuesta dos casillas que no se usan y ahorra estar restando. Y `[False] * (n + 1)` es seguro, a diferencia de la tabla de la sección anterior, porque `False` es inmutable: da igual que las veintiséis casillas compartan el mismo, porque nadie lo va a modificar en el lugar, solo reemplazarlo.

Con eso la criba cabe en doce líneas:

```
def criba(n):
    """Devuelve la lista de los primos hasta n, por el método de Eratóstenes."""
    compuesto = [False] * (n + 1)
    primos = []

    for candidato in range(2, n + 1):
        if compuesto[candidato]:
            continue

        primos.append(candidato)

        for multiplo in range(candidato * candidato, n + 1, candidato):
            compuesto[multiplo] = True

    return primos

print(criba(25))
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23]
```

Léela contra la figura 4. `compuesto[k]` es el tachón del número `k`. El ciclo de afuera recorre los candidatos; si uno viene tachado, `continue` salta a la vuelta siguiente. Si no viene tachado es primo, se anota con `append`, y el ciclo de adentro tacha sus múltiplos.

Dos detalles que pagan la pena de leerlos despacio. El ciclo de adentro empieza en `candidato * candidato`, no en `candidato * 2`, porque cualquier múltiplo menor que el cuadrado ya tiene un factor más

chico que lo tachó antes: cuando llegamos al 5, el 10 y el 15 y el 20 ya están tachados por el 2 y por el 3. Y el paso del `range` es `candidato`, que es lo que hace que el ciclo salte de múltiplo en múltiplo sin comprobar nada.

Los dos métodos, medidos

El mismo trabajo por los dos caminos: contar los primos que hay hasta un millón.

```
import math
import time

def es_primo(n):
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    for d in range(3, int(math.sqrt(n)) + 1, 2):
        if n % d == 0:
            return False

    return True

def contar_primos(hasta):
    total = 0

    for k in range(2, hasta + 1):
        if es_primo(k):
            total += 1

    return total

inicio = time.perf_counter()
print(contar_primos(1000000), "primos, uno por uno")
print(f"{time.perf_counter() - inicio:.2f} s")

inicio = time.perf_counter()
print(len(criba(1000000)), "primos, tachando")
print(f"{time.perf_counter() - inicio:.2f} s")
```

```
78498 primos, uno por uno
1.67 s
78498 primos, tachando
0.14 s
```

Segundos contra décimas de segundo, y las dos respuestas son la misma. Pero los segundos no son un buen instrumento: dependen de la máquina, de lo que esté haciendo el sistema operativo y de la versión de Python. Corre ese bloque dos veces y te van a salir números distintos.

Lo que no cambia es **cuántas operaciones hace cada método**. El de uno en uno divide, así que se cuentan sus divisiones. La criba tacha, así que se cuentan sus tachones. Esos dos números son exactos y siempre los mismos:

```
import math

def divisiones_hasta(n):
    """Cuenta las divisiones que hace el método de uno en uno para llegar a n."""
    total = 0
```

```
    for k in range(2, n + 1):
        for d in range(2, int(math.sqrt(k)) + 1):
            total += 1

            if k % d == 0:
                break

    return total

def tachones_hasta(n):
    """Cuenta los tachones que hace la criba para llegar a n."""
    compuesto = [False] * (n + 1)
    total = 0

    for candidato in range(2, n + 1):
        if compuesto[candidato]:
            continue

        for multiplo in range(candidato * candidato, n + 1, candidato):
            compuesto[multiplo] = True
            total += 1

    return total

topes = [100000, 300000, 900000]
divisiones = []
tachones = []

for tope in topes:
    divisiones.append(divisiones_hasta(tope))
    tachones.append(tachones_hasta(tope))

    print(f"hasta {tope:>7}: {divisiones[-1]:>10} divisiones"
          f"   contra {tachones[-1]:>8} tachones")
```

```
hasta  100000:    2745694 divisiones   contra   193078 tachones
hasta  300000:   12582930 divisiones   contra   607825 tachones
hasta  900000:   58461003 divisiones   contra  1902617 tachones
```

Las dos cuentas se van guardando en sendas listas, que es justo lo de hoy: no sabemos de antemano cuántos topes vamos a probar.

Mira esa tabla por columnas y no por filas. Cada línea multiplica el tope por tres. Los tachones de la criba también se multiplican por tres, más o menos: hacer el triple de trabajo para resolver el triple de números es lo mejor que se puede esperar. Las divisiones del otro método se multiplican por casi cinco.

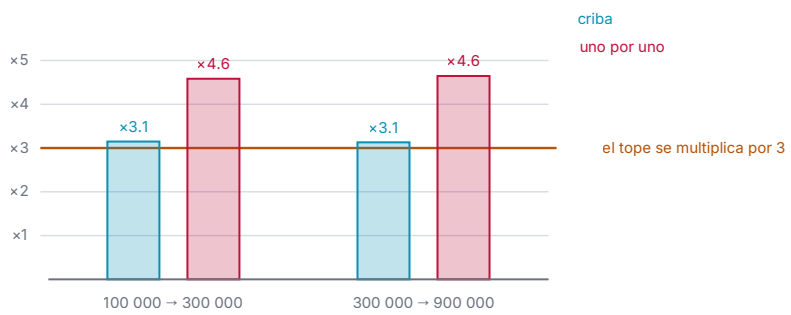


Figura 5. Lo que se multiplica cada cuenta cuando el tope se multiplica por tres. La criba se mantiene en el triple; el otro método se va a casi el quíntuple.

Ahí está la diferencia, y no es que un programa sea diez veces más lento que el otro. Es que **la distancia entre los dos se ensancha cada vez que crece `n`.** Un factor de diez se paga comprando una computadora mejor. Esto no.

Eso se llama **orden de crecimiento**, es el tema de la conferencia ocho, y esta tabla se va a volver a hacer allí con nombres y con fórmulas. Por ahora quédate con la razón intuitiva de por qué la criba gana. El método de uno en uno empieza de cero con cada número y tira a la basura todo lo que averiguó. La criba hace lo contrario: cada primo que encuentra lo gasta inmediatamente en descartar de golpe a muchos otros, así que cada respuesta que consigue le abarata las siguientes.

Dejar de recalcular y empezar a recordar es la idea de la última conferencia del semestre, y para recordar hace falta dónde guardar, que es lo de hoy.

9. Resumen

- Una lista guarda muchos valores con un solo nombre. Se escribe con corchetes y su tamaño se pide con `len`.
- Los índices empiezan en cero y el último es `len - 1`. Los negativos cuentan desde el final. Pasarse da `IndexError`.
- `for nota in notas` recorre los elementos y es la forma normal. Usa el índice solo si lo necesitas, y entonces `enumerate`.
- Las listas son mutables: `append`, `pop`, `insert`, `remove`, `sort`.
- `b = a` no copia la lista, le pone otro nombre. Para copiar, `list(a)` o `a[:]`.
- Una función que recibe una lista puede modificarla, y el cambio se ve desde fuera.
- Una rebanada `a[i:j]` construye una lista nueva.
- `sum`, `max`, `min` y `sorted` existen, pero conviene saber escribirlos.
- Una tupla es una secuencia que no se puede modificar. Lo que la construye es la coma, y sirve para unos pocos valores donde cada posición significa algo: un punto, un nombre con su nota.
- `x, y = punto` desempaqueta una tupla. `for nombre, nota in grupo` y `for i, nota in enumerate(notas)` hacen lo mismo en cada vuelta, `return a, b` devuelve dos valores y `a, b = b, a` los intercambia.
- Una lista de listas es una tabla: `tabla[fila][columna]`, y se recorre con dos ciclos anidados.
- `[[0] * 3] * 2` no construye dos filas. Constrúyelas en un ciclo. `[False] * n` sí está bien, porque `False` es inmutable.
- Una lista es lo que permite guardar tantos resultados intermedios como haga falta sin saber cuántos son al escribir el programa. La criba de Eratóstenes vive entera de eso.

Ejercicios

1. Escribe `segundo_mayor(lista)` que devuelva el segundo elemento más grande, sin ordenar la lista y recorriéndola una sola vez. Piensa qué debe pasar con `[5, 5, 3]`.

2. Escribe `minimo_y_maximo(lista)` que devuelva una tupla con el menor y el mayor elemento, sin usar `min` ni `max` y recorriendo la lista una sola vez.

3. Escribe `invertir(lista)` que devuelva una lista nueva con los elementos al revés, sin usar `[::-1]` ni `reverse`. Después escribe `invertir_en_el_lugar(lista)` que no devuelva nada y modifique la que recibió, intercambiando el primero con el último, el segundo con el penúltimo, y así. ¿Hasta dónde tiene que llegar el ciclo?

4. Escribe `sin_repetidos(lista)` que devuelva una lista nueva con los elementos de la original, en el mismo orden, pero sin repeticiones.

5. Escribe `es_capicua(lista)` que diga si la lista se lee igual al derecho y al revés. Resuélvelo sin construir ninguna lista nueva.

6. Con una tabla de números como la de la sección 7, escribe `transpuesta(tabla)` que devuelva una tabla nueva donde las filas son las columnas de la original. Comprueba que la transpuesta de la transpuesta es la tabla de partida.

7. Vuelve a escribir `contar_gemelos(n)` del ejercicio 5 de la clase pasada, ahora sobre la lista que devuelve `criba`. Cronometra las dos versiones con `n` de un millón y compara el resultado, que tiene que ser el mismo número.

8. Cuenta los primos menores que un millón con la criba, y después prueba con diez millones y con cien millones. En algún punto el programa deja de ser lento y pasa a ser imposible. Di en qué punto y por qué; la respuesta no tiene que ver con el tiempo sino con lo que ocupa `compuesto`. Estima cuántos bytes hacen falta por casilla.

9. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

   (a)

   ```
   a = [1, 2, 3]
   b = a
   c = a[:]
   a[0] = 99
   print(b[0], c[0])
   ```

   (b)

   ```
   def f(lista):
       lista = [0, 0, 0]

   def g(lista):
       lista[0] = 0

   x = [1, 2, 3]
   f(x)
   print(x)
   g(x)
   print(x)
   ```

   (c)

   ```
   numeros = [1, 2, 4, 3]

   for n in numeros:
       if n % 2 == 0:
           numeros.remove(n)

   print(numeros)
   ```

   El tercero quiere eliminar los pares y no lo consigue. Explica por qué, y arréglalo. Después corre el mismo programa con `[1, 2, 3, 4]`, donde sí da la respuesta correcta, y explica por qué esa entrada esconde el error. Es la clase de coincidencia que hace que un programa roto pase las pruebas.